

ALPHAQUANT: LLM-DRIVEN AUTOMATED ROBUST FEATURE ENGINEERING FOR QUANTITATIVE FINANCE

Kamer Ali Yuksel & Hassan Sawaf

aiXplain Inc., San Jose, CA, USA

{kamer, hassan}@aixplain.com

ABSTRACT

Feature engineering is critical to predictive modeling, transforming raw data into meaningful features that enhance performance. However, traditional feature engineering is labor-intensive and prone to biases, while automated methods often lack robustness and interpretability. This paper introduces a novel framework that combines large language models (LLMs) with evolutionary optimization to automate robust feature discovery. The framework integrates LLMs for domain specific feature generation and a rigorous evaluation loop using machine-learning models hyper-tuned with time-series cross-validation on historical asset performance to ensure their robustness. The key contributions: (1) an LLM-powered system for generating domain-relevant, interpretable feature extraction functions, (2) an evolutionary illumination process that iteratively refines the feature-set based on importance scores from hyper-tuned models, and (3) empirical validation on financial data demonstrating significant improvements in predictive accuracy and feature robustness. The results highlight the potential of LLMs to revolutionize feature engineering, paving the way for interpretable machine-learning models. The set of discovered features is open-sourced for the reproducibility of results.

1 INTRODUCTION

Feature engineering transforms raw data into meaningful variables that enhance model performance, and has long been a key focus in predictive modeling. Traditional feature engineering, however, relies heavily on domain expertise and manual iteration, which is labor-intensive, time-consuming, and prone to errors and biases. Automated feature engineering methods systematically generate features by applying mathematical operations to raw variables to address these challenges but often fail to account for domain-specific nuances, resulting in features that lack robustness and interoperability. The challenges are especially pronounced in domains like finance, where data is noisy, non-stationary, and influenced by extreme events. Features must generalize across time and assets, requiring sophisticated evaluation mechanisms to ensure robustness. Traditional methods often fail to capture domain-specific insights or ensure feature robustness across assets or market regimes.

Recent advancements in Large Language Models (LLMs) have opened new possibilities for automating feature discovery. LLMs can generate interpretable feature extraction functions tailored to domain requirements, synthesizing knowledge from mathematical principles and financial literature. When paired with evolutionary optimization frameworks, LLMs can enable iterative refinement of features to maximize robustness and performance. To ensure robustness, features must be evaluated using advanced cross-validation techniques. Time-series cross-validation ensures that features generalize across temporal data splits, while leave-one-out cross-validation evaluates feature performance across assets. By combining these evaluation mechanisms with LLM-driven feature generation and optimization, This work represents a significant advancement in feature engineering.

LLMs, such as GPT-4 and LLama, offer an innovative approach to automating feature generation. These models synthesize domain-specific knowledge and generate creative solutions informed by vast corpora. When coupled with quality-diversity optimization—a paradigm that emphasizes exploration and refinement—LLMs can enable robust, interpretable, and scalable feature engineering. This paper introduces a novel framework integrating LLM-driven feature generation with hyper-

tuned model evaluation and iterative optimization. Using financial data as a case study, we demonstrate its ability to automate feature engineering while ensuring robustness through cross-validation.

2 RELATED WORK

The advent of large language models has spurred a new line of research in automated feature engineering. Researchers have begun to harness LLMs’ few-shot and in-context learning abilities to generate candidate feature transformations. For instance, Zhang et al. introduced a “Dynamic and Adaptive Feature Generation” method that uses LLMs to generate new feature extraction functions in a controlled, interpretable manner Zhang et al. (2024). Similarly, Gong et al. proposed an “Evolutionary Large Language Model for Automated Feature Transformation,” in which an LLM is teamed with evolutionary search strategies to explore the vast space of possible feature-transformation sequences Gong et al. (2024). In these works the LLM serves not merely as a language generator but as a domain knowledge-integrator, capable of leveraging textual and mathematical cues to propose creative and contextually relevant features. In addition, several surveys and frameworks have begun to review how LLMs can be applied throughout machine learning workflows—including data preprocessing, feature engineering, and model selection Gu et al. (2024). While these works focus broadly on automating ML pipelines, our contribution zeroes in on the feature engineering stage, combining LLM-driven creative generation with quality-diversity optimization to iteratively refine and validate feature sets. Taken together, these strands of research motivate our framework: by coupling the creative and domain-informed capabilities of LLMs with the robust exploration provided by quality-diversity optimization, our approach offers a scalable, interpretable, and high-performing solution for automated feature engineering in challenging domains such as finance.

Recent advances in quantitative finance have also shifted the focus from manually crafted heuristics to data-driven, automated pipelines for discovering predictive signals. Two complementary strands have emerged: one in which LLMs are deployed as alpha miners within trading agents, and another where LLMs facilitate automated feature engineering to transform raw data into robust, interpretable inputs for predictive models. Early work has demonstrated the promise of LLMs in financial applications by positioning them either as direct signal generators (i.e., trading agents) or as alpha miners that produce candidate factors for downstream trading systems. In this paradigm, architectures such as QuantAgent and AlphaGPT employ an inner-outer loop strategy: an LLM first generates candidate alpha expressions based on high-level trader ideas, and an outer loop evaluates these candidates against historical data to iteratively refine the factors (Wang, 2023; 2024). This circumvents the need for manual feature extraction, enabling agents to adapt dynamically to evolving market conditions and synthesize insights from textual and numerical financial data. Lastly, research by Kou et al. (2024) has integrated LLMs within multi-agent architectures to mine and validate alphas across diverse modalities where an evaluation process back-tests candidate alphas under various risk preferences and market scenarios, dynamically selecting and weighting the most robust signals.

Together, these strands of research illustrate a paradigm shift in financial feature engineering and signal discovery. The transition from rigid, expert-driven approaches to flexible, LLM-powered frameworks enables the extraction of richer, more adaptive predictive signals from complex, multi-modal data sources. By leveraging LLMs’ creative and domain-informed capabilities, recent work offers a scalable, interpretable, and high-performing solution for automated feature engineering in challenging domains such as finance. They demonstrate that combining LLM-driven feature discovery with ensemble evaluation methods can lead to more adaptive and resilient quantitative strategies. Moreover, in the financial domain, Arian et al. (2024) introduce a Stability-Weighted Ensemble Feature Importance method designed specifically for financial applications, demonstrating how feature importance can be enhanced by combining stability selection with ensemble learning.

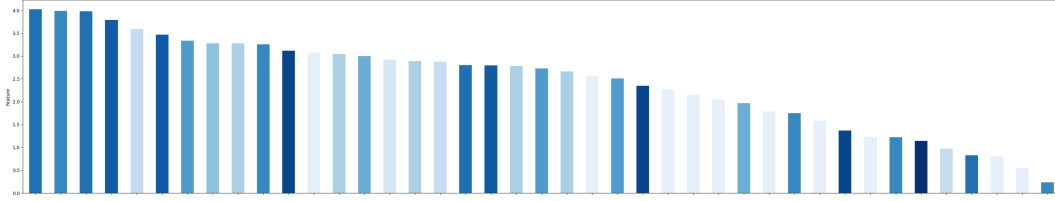


Figure 1: Feature importance where dark colors indicate features being discovered in later iterations. The plot shows that the proposed method is able to discover more important features over iterations.

3 METHODOLOGY

Our methodology implements an iterative feature engineering workflow for financial datasets by combining automated feature generation driven by large language models (LLMs), parallelized feature extraction and evaluation, and iterative model optimization via AutoML. The goal is to extract novel, predictive financial features from time-series log-return data to enhance risk-adjusted performance measures, such as the Sharpe ratio. The system integrates the following components:

- **LLM-Driven Feature Synthesis:** An LLM is prompted with a set of few-shot examples (e.g., basic statistical functions) to generate new candidate feature extraction functions in PyTorch by proposing innovative features inspired by established financial literature.
- **Parallelized Feature Extraction:** Newly generated feature functions are executed parallel for each asset in a global context using Python’s concurrency libraries. Each feature is validated for correct dimensionality, absence of NaN/Inf values, and sufficient variance.
- **AutoML-Based Evaluation:** Extracted features are evaluated using an AutoML framework (FLAML) with LightGBM as the regression estimator. A state graph orchestrates the cyclic process of feature extraction, evaluation, ranking, and new candidate generation. Each iteration refines the feature set based on previous performance and diagnostic metrics.

3.1 FEATURE GENERATION USING LLM

A key innovation in our approach is the use of LLMs to propose novel feature extraction functions:

- **Few-Shot Prompting:** A set of basic PyTorch functions (e.g., `mean`, `variance`, `std`, etc.) is provided as initial few-shot examples and updated as better features are discovered.
- **Error-Aware Generation:** The prompt includes a list of previously eliminated functions and a log of execution errors to steer the LLM away from redundant or erroneous features.
- **Code Extraction and Compression:** The generated code is post-processed using Abstract Syntax Trees (AST) to extract valid function definitions and to remove extraneous comments and docstrings, ensuring that only the essential code is retained.

3.2 AUTOML TRAINING AND FEATURE RANKING

The extracted feature set is subsequently assessed with cross-validation using a AutoML framework:

- **AutoML Framework:** FLAML is used to train a regression model (with LightGBM as the estimator) on the extracted feature matrix. The hyper-tuning is always warm-started.
- **Performance Metrics:** The model training is carried out using the standard R^2 objective. Other performance measures include Mean Absolute Error (MAE), Spearman correlation, and Normalized Discounted Cumulative Gain (nDCG) at specified quantile cutoffs.
- **Feature Importance Extraction:** Post-training, feature importances are extracted from the LightGBM model, which are used to rank the features and drive the selection process.

3.3 ITERATIVE WORKFLOW FOR FEATURE ENGINEERING

The overall system is organized as a cyclic state graph that defines a multi-stage iterative workflow:

1. **Feature Extraction:** The current candidate feature functions are applied to the training data to produce a feature matrix. The top- k features are selected based on feature importances from the AutoML model hyper-tuned with time-series cross-validation.
2. **Feature Selection:** Features that are underperforming or redundant are eliminated, and the LLM keeps generating new candidate features. The new generation is informed by previous iteration metadata, including error logs and eliminated functions.

4 EXPERIMENTS

The experiments were conducted on a real-world financial dataset. Time-series cross-validation was employed to validate and rank features as they evolve. The dataset, comprising 15 years of historical data from 3,246 US stocks and ETFs, was split into rolling windows with a 50% overlap after separating 10% of the data for the out-of-sample test (1.5 years). In our experiments, the test window size is set to 10% of the total time steps, while the training window is three times larger. The blind test set included recent periods of extreme market stress, such as the 2020 COVID-19 market crash, highlighting the robustness and stability of the discovered features under high-volatility conditions. The AutoML training has been performed with the Mean-Absolute-Error (MAE) objective from cross-validation where folds were organized as grouped rolling windows based on their start date. An example set of discovered features is open-sourced for reproducibility of experimental results.

4.1 WORKFLOW CONFIGURATION

- **Initial Feature Set:** Ten basic statistical functions are provided as seed feature candidates.
- **LLM Integration:** LLM is utilized to generate novel candidate features. The prompts are designed to encourage the synthesis of advanced risk-return indicators.
- **Iteration Control:** The state graph orchestrates three main tasks in each iteration:
 1. *Feature Extraction and Scoring:* New candidate features are applied to the training data and validated.
 2. *Feature Ranking and Selection:* AutoML (via FLAML and LightGBM) is used to evaluate and rank features based on predictive performance.
 3. *Candidate Generation:* New feature candidates are generated by the LLM, with redundant or underperforming features eliminated.
- **Time Budget:** Each AutoML training session is allocated a time budget of 60 seconds, with warm-starting from previous configurations to accelerate convergence.

We assess the predictive performance of models trained with discovered feature sets using:

- **Mean Absolute Error (MAE):** Evaluates the average magnitude of errors in predictions.
- **Spearman Correlation:** Assesses the monotonic relationship between the predicted and future Sharpe ratios.
- **Normalized Discounted Cumulative Gain (nDCG):** Computed at different quantile cut-offs (nDCG@Q1 and nDCG@Q4) to measure the quality of the feature ranking.

4.2 RESULTS AND ANALYSIS

At each workflow iteration, the AutoML model hyper-tuned with newly discovered features is also evaluated on the blind test set. The experimental results demonstrate the efficacy of our iterative feature engineering framework. Incorporating LLM-generated features leads to a more diverse and predictive feature set. Iterative refinement, combined with AutoML and feature importance ranking, consistently improves predictive performance. Overall, our approach improves forecasting performance (as measured by MAE, Spearman, and nDCG) and provides interpretable insights into the role of individual features in risk-adjusted performance evaluation. The experiments validate the

proposed methodology as an effective dynamic financial feature engineering tool. Integrating LLM-driven feature synthesis, robust feature extraction, and iterative AutoML evaluation forms a comprehensive framework that can adapt to the evolving nature of financial data and market conditions.

- **Improved Predictive Accuracy:** The iterative process leads to progressive reductions in MAE. For example, initial iterations with basic features yielded higher MAE compared to later iterations where novel LLM-generated features were incorporated.
- **Enhanced Feature Ranking:** The top- k features are selected based on feature importances extracted from LightGBM, or optionally obtained from SHAP values.
- **Ranking Correlation Analysis:** Spearman and nDCG values indicate an increasingly stronger monotonic relationship among feature-driven predictions and blind Sharpe ratios.

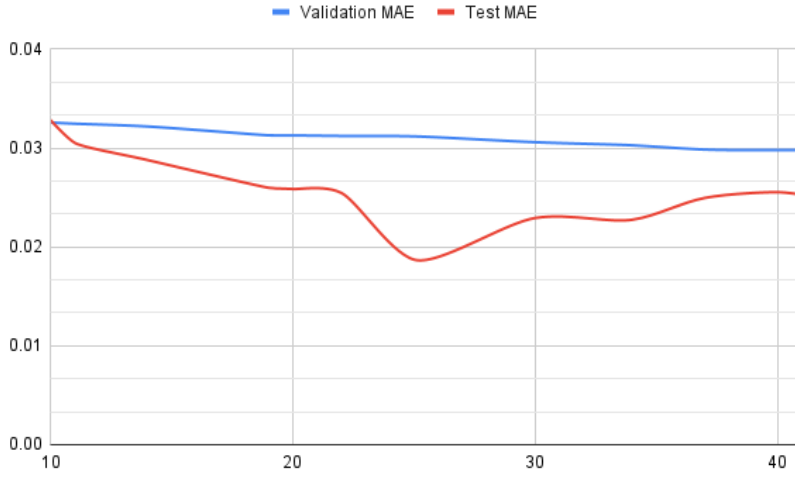


Figure 2: Mean-absolute-error (MAE) of predictions in validation and test-set with more features. The plot shows a tendency to decrease validation and test errors as more features are discovered.

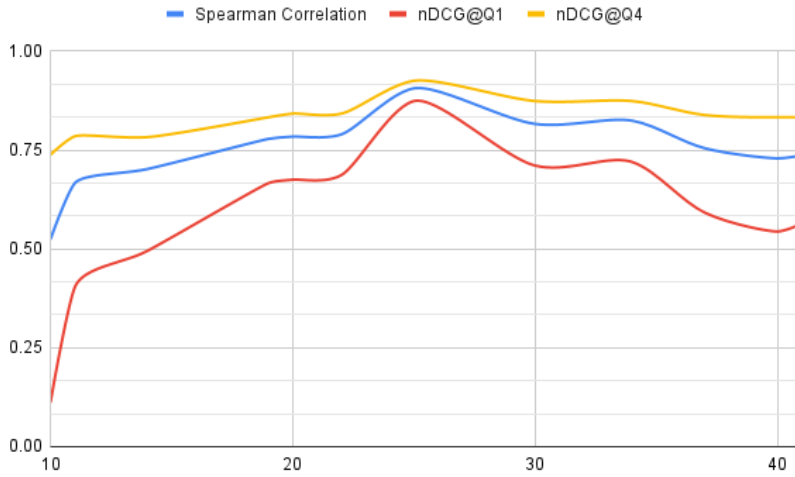


Figure 3: Spearman and nDCG of model predictions vs test-set Sharpe ratios with more features. The plot shows the increased model’s capability to rank future asset performance over iterations.

4.3 CONCLUSION

By integrating LLM-driven feature synthesis with rigorous extraction, evaluation, and iterative refinement, our methodology provides an end-to-end automated framework for financial feature engineering. AutoML with time-series cross-validation ensures that only the most robust predictive features are retained. The proposed method is well-suited for financial environments, where the LLM-driven discovery of novel features can significantly enhance the forecasting performance.

REFERENCES

- HR Arian, SA Mousavizade, and LA Seco. Stability-weighted ensemble feature importance for financial applications. Working Paper Abstract ID: 4905824, SSRN, 2024. URL https://papers.ssrn.com/sol3/papers.cfm?abstract_id=4905824. Accessed: February 03, 2025.
- Nanxu Gong, Chandan K. Reddy, Wangyang Ying, and Yanjie Fu. Evolutionary large language model for automated feature transformation, 2024. Preprint.
- Yang Gu, Hengyu You, Jian Cao, and Muran Yu. Large language models for constructing and optimizing machine learning workflows: A survey, 2024. Survey.
- Zhizhuo Kou, Holam Yu, Jingshu Peng, and Lei Chen. Automate strategy finding with llm in quant investment. *arXiv preprint arXiv:2409.06289*, 2024.
- et al. Wang. Alphagpt: A human in the loop framework for alpha mining. <https://arxiv.org/abs/2408.06361v1>, 2023. Details provided in the survey.
- et al. Wang. Quantagent: Llm as an alpha miner in financial trading. <https://arxiv.org/abs/2408.06361v1>, 2024. Extracted from survey discussion.
- Xinhao Zhang, Jinghan Zhang, Banafsheh Rekabdar, Yuanchun Zhou, Pengfei Wang, and Kunpeng Liu. Dynamic and adaptive feature generation with llm. *arXiv preprint arXiv:2406.03505*, 2024.

A APPENDIX

A.1 SYSTEM PROMPT

You are an expert in financial metrics and feature engineering. Generate PyTorch feature extraction code for a financial dataset.

You aim to engineer predictive features by proposing interestingly novel statistical risk-return indicators. USE CRITICAL THINKING!

You are deeply familiar with statistical time-series features from literature, and encouraged to draw inspiration from publications. Use the knowledge from examples and inspiration from academic literature to propose new financial analysis feature implementations.

Each feature should be unique, interpretable, and useful for predicting Sharpe Ratios. Avoid redundancy with the examples provided. When features are applied to the past log-returns of assets, obtained indicators should help predicting assets' future sharpe-ratios.

A.2 USER PROMPT

Example features (PyTorch code snippets) are below. You should implement novel more advanced (better) features that are complementary.

{top_k_examples}

Generate {num_features} new feature extraction functions. Give concise names to the functions that summarize the feature extracted. It must take log_returns (torch.Tensor) as the only input parameter that doesn't have any default value defined in the function body. Prefer using tensor operations. Return only the PyTorch implementation of each feature without any additional markdown or comments.

Avoid extracting following features that have been previously tried but were redundant, come up with better ones. THINK OUTSIDE THE BOX!

{eliminated_list}

Below are the execution errors encountered in previous attempts. Avoid similar mistakes in future implementations: \n{previous_errors}